RB-104: Log Shipping Pipeline Broken

Field	Value
Service	Log pipeline: Filebeat, Kafka topic logs-raw, Logstash, Elasticsearch es-logs
Severity	SEV-3 (SEV-2 if security or audit logs are missing)
Owner	Platform Ops team
Last reviewed	2026-10-12

1. Symptoms

- Kibana shows a gap: no new documents in the logs-app-* index pattern for more than 10 minutes while the hosts are healthy.
- Alert LogIngestRateLow fires when Elasticsearch indexes fewer than 2,000 docs/s for 10 minutes (normal daytime rate is 15,000 to 25,000).
- Alert LogstashConsumerLagHigh fires when consumer group logstash-indexer lags more than 5,000,000 messages on logs-raw.

If the gap is limited to log-01, log-02 or log-03 and their /var/log is filling up, use RB-101 instead.

2. Pipeline Overview

Logs flow through four hops. Find the first hop where data stops; everything after it is a consequence.

Hop	Component	Where it runs	Key health signal
1	Filebeat 8.x	every app host and node, as DaemonSet filebeat in namespace logging	libbeat.output.events.acked rising
2	Kafka topic logs-raw	kafka-logs-01..03, 24 partitions, retention 48 h	end offsets rising
3	Logstash	Deployment logstash-indexer, 6 replicas, namespace logging	pipeline events out rising
4	Elasticsearch	cluster es-logs, 9 data nodes	indexing rate, cluster status green

3. Diagnosis

3.1 Hop 1: is Filebeat publishing?

```
kubectl -n logging get ds filebeat
kubectl -n logging logs ds/filebeat --tail=100 | grep -iE 'error|backoff|kafka'
curl -s localhost:5066/stats | jq '.libbeat.output.events'
```

On a host, acked should grow and failed stay flat. Kafka publish failed ... broker not available points to hop 2. Filebeat buffers up to 4096 events in memory, then stops reading files; it does not drop data.

3.2 Hop 2: is Kafka receiving data?

```
kafka-run-class kafka.tools.GetOffsetShell --bootstrap-server kafka-logs-01:9092 --topic logs-raw
kafka-consumer-groups --bootstrap-server kafka-logs-01:9092 --describe --group logstash-indexer
```

Run the offset command twice, 60 s apart. If end offsets do not move, the problem is hop 1. If end offsets move but LAG grows, the problem is hop 3 or 4.

3.3 Hop 3 and 4: Logstash and Elasticsearch

```
kubectl -n logging exec deploy/logstash-indexer -- curl -s localhost:9600/_node/stats/pipelines | jq '.pipe
curl -s http://es-logs:9200/_cluster/health?pretty
curl -s http://es-logs:9200/_cat/thread_pool/write?v
```

Rejections in the write thread pool, or status red, mean Elasticsearch is pushing back. Logstash then slows down; this is backpressure, not a Logstash bug.

4. Remediation

4.1 Backpressure from Elasticsearch

1. If the cluster has a read-only block from the flood-stage disk watermark, free space first, then clear it with PUT logs-app-*/_settings and index.blocks.read_only_allow_delete: null.
2. If write rejections continue, reduce Logstash to 4 replicas to relieve Elasticsearch: kubectl -n logging scale deploy logstash-indexer --replicas=4.
3. Kafka retains 48 h of logs, so Logstash will catch up once Elasticsearch recovers; no data is lost if this is fixed within that window.

4.2 Filebeat stuck on a host

Restart the pod on the affected node: kubectl -n logging delete pod <FILEBEAT_POD>. Filebeat resumes from its registry in /var/lib/filebeat/registry and re-reads only lines not yet acknowledged by Kafka.

Warning: do not delete the Filebeat registry (/var/lib/filebeat/registry) to "unstick" Filebeat. Without the registry it re-reads every file from the start and floods Kafka and Elasticsearch with days of duplicate logs.

4.3 Logstash consumers stuck

If Logstash pods are running but the consumer group shows no active members, restart them with kubectl -n logging rollout restart deploy logstash-indexer. Do not reset the consumer group offsets; that either skips logs or re-indexes them twice.

5. Verification

- Kibana shows new documents in logs-app-* within the last minute.

- logstash-indexer lag is falling by at least 100,000 messages per minute.
- LogIngestRateLow and LogstashConsumerLagHigh have resolved.

6. Escalation

Level	Who	Contact
L1	On-call SRE	#ops-oncall
L2	Platform Ops on-call	PagerDuty: platform-ops
L3	Streaming team on-call	PagerDuty: streaming (Kafka broker problems on kafka-logs-0x)